Module - 4 (Object Design)

Object Design: Overview of Object design, Combining the three models, Designing algorithms, Design optimization, Implementation of control, Adjustment of inheritance, Design of association, Object representation, Physical packaging, Documenting design decisions -Comparison of methodologies.

Object Design

- The object design phase determines the full definitions of the classes and associations used in the implementation, as well as the interfaces and algorithms of the methods used to implement operations.
- The object design phase adds internal objects for implementation and optimizes data structures and algorithms.

Overview of Object Design

- During object design, the designer carries out the strategy chosen during the system design and fleshes out the details.
- There is a shift in emphasis from application domain concepts toward computer concepts.
- The objects discovered during analysis serve as the skeleton of the design, but the object designer must choose among different ways to implement them with an eye toward minimizing execution time, memory and other measures of cost.

- The operations identified during the analysis must be expressed as algorithms, with complex operations decomposed into simpler internal operations.
- The classes, attributes and associations from analysis must be implemented as specific data structures.
- New object classes must be introduced to store intermediate results during program execution and to avoid the need for recomputation.
- Optimization of the design should not be carried to excess, as ease of implementation, maintainability, and extensibility are also important concerns.

Steps of Object Design:

During object design, the designer must perform the following steps:

1. Combining the three models to obtain operations on classes.
2. Design algorithms to implement operations.
3. Optimize access paths to data.
4. Implement control for external interactions
5. Adjust class structure to increase inheritance.
6. Design associations.
7. Determine object representation.
8. Package classes and associations into modules.

Combining the three models to obtain operations on classes.

- After analysis, we have object, dynamic and functional model, but the object model is the main framework around which the design is constructed.
- The object model from analysis may not show operations.
- The designer must convert the actions and activities of the dynamic model and the processes of the functional model into operations attached to classes in the object model.

- Each state diagram describes the life history of an object.
- A transition is a change of state of the object.
- We can associate an operation with each event received by an object.
- In the state diagram, the action performed by a transition depends on both the event and the state of the object.
- Therefore, the algorithm implementing an operation depends on the state of the object.

- If the same event can be received by more than one state of an object, then the code implementing the algorithm must contain a case statement dependent on the state.
- An event sent by an object may represent an operation on another object .
- Events often occur in pairs , with the first event triggering an action and the second event returning the result on indicating the completion of the action.
- An action or activity initiated by a transition in a state diagram may expand into an entire DFD in the functional model .

- The network of processes within the DFD represents the body of an operation.
- The flows in the diagram are intermediate values in operation.
- The designer convert the graphic structure of the diagram into linear sequence of steps in the algorithm .
- The process in the DFD represent suboperations.
- Some of them, but not necessarily all may be operations on the original target object or on other objects.

Determine the target object of a suboperation as follows:

* If a process extracts a value from input flow then input flow is the target.
* Process has input flow or output flow of the same type, input output flow is the target.
* Process constructs output value from several input flows, then the operation is a class operation on the output class.
* If a process has input or an output to data store or actor, data store or actor is a target of the process.

Designing algorithms

- Each operation specified in the functional model must be formulated as an algorithm.
- The analysis specification tells what the operation does from the view point of its clients, but the algorithm shows how it is done.
- An algorithm may be subdivided into calls on simpler operations, and so on recursively, until the lowest-level operations are simple enough to implement directly without refinement .

- The algorithm designer must decide on the following:
  - Choosing algorithms that minimize the cost of implementing operations
  - Choosing Data Structures appropriate to the algorithm
  - Define new Internal Classes and Operations
  - Assign Responsibility for Operations to appropriate Classes

1.Choosing algorithms

- Many operations are simple enough that the specification in the functional model already constitutes a satisfactory algorithm because the description of what is done also shows how it is done.
- Many operations simply traverse paths in the object link network or retrieve or change attributes or links.
- Non trivial algorithm is needed for two reasons:
  a) To implement functions for which no procedural specification is given
  b) To optimize functions for which a simple but inefficient algorithm serves as definition.

- Some functions are specified as declarative constraints without any procedural definition.
- The essence of most geometry problems is the discovery of appropriate algorithms and the proof that they are correct.
- Most functions have simple mathematical or procedural definitions.
- Often the simple definition is also the best algorithm for computing the function or else is also so close to any other algorithm that any loss in efficiency is the worth the gain in clarity.
- In other cases, the simple definition of an operation would be hopelessly inefficient and must be implemented with a more efficient algorithm.

- Consider the algorithm for search operation .
- A search can be done in two ways like binary search (which performs log n comparisons) and a linear search (which performs n/2 comparisons).
- Suppose our search algorithm is implemented using linear search , which needs more comparisons.
- It would be better to implement the search with a much efficient algorithm like binary search.
- Considerations in choosing among alternative algorithm include:

a) Computational Complexity:

- It is essential to think about complexity i.e. how the execution time (memory) grows with the number of input values.
- eg: For a bubble sort algorithm, time ∞ n^2
- Most other algorithms, time ∞ n log n

b) Ease of implementation and understandability:

- It is worth giving up some performance on non critical operations if they can be implemented quickly with a simple algorithm.

- **c) Flexibility:**
- Most programs will be extended sooner or later. A highly optimized algorithm often sacrifices readability and ease of change.
- One possibility is to provide two implementations of critical applications, a simple but inefficient algorithm that can be implemented quickly and used to validate the system, and a complicated but efficient algorithm whose correct implementation can be checked against the simple one.

d) Fine Timing the Object Model:

- We have to think, whether there would be any alternatives, if the object model were structured differently.

2.Choosing Data Structures

- Choosing algorithms involves choosing the data structures they work on.
- We must choose the form of data structures that will permit efficient algorithms.
- The data structures do not add information to the analysis model, but they organize it in a form convenient for the algorithms that use it.
- Many implementation data structures are instances of container classes such as arrays, lists, queues, stacks, dictionaries etc.

3.Defining Internal Classes and Operations

- During the expansion of algorithms, new classes of objects may be needed to hold intermediate results.
- New, low level operations may be invented during the decomposition of high level operations
- A complex operation can be defined in terms of lower level operations on simpler objects.
- These lower level operations must be defined during object design because most of them are not externally visible.
- Some of these operations were found from "shopping –list" operations that were identified during analysis as being potentially useful.

- There is a need to add new internal operations as we expand high level functions.
- When you reach this point during the design phase, you may have to add new classes that were not mentioned directly in the client's description of the problem.
- These low-level classes are the implementation elements out of which the application classes are built.

4.Assigning Responsibility for Operations

- Many operations have obvious target objects, but some operations can be performed at several places in an algorithm
- Such operations are often part of a complex high-level operation with many consequences.
- Assigning responsibility for such operations can be frustrating.
- When a class is meaningful in the real world, then the operations on it are usually clear.
- During implementation, internal classes are introduced.

- *How do you decide what class owns an operation?*
- When only one object is involved in the operation, tell the object to perform the operation.
- When more than one object is involved, the designer must decide which object plays the lead role in the operation.
- For that, ask the following questions:
  - Is one object acted on while the other object performs the action?
    - It is best to associate the operation with the target of the operation, rather than the initiator.
  - Is one object modified by the operation, while other objects are only queried for the information they contain?
    - The object that is changed is the target.

- Looking at the classes and associations that are involved in the operation, which class is the most centrally-located in this sub network of the object model?
  - If the classes and associations form a star about a single central class, it is the target of the operation.
- If the objects were not software, but the real world objects represented internally, what real world objects would you push, move, activate or manipulate to initiate operation?

- Assigning an operation within a generalization hierarchy can sometimes be difficult because the definitions of the subclasses within the hierarchy are often fluid and can be adjusted during design .
- It is common to move an operation up and down in the hierarchy during design, as its scope is adjusted.

Design Optimization

- The basic deign model uses the analysis model as the framework for implementation .
- The analysis model captures the logical information about the system, while the design model must add details to support efficient information access.
- The inefficient but semantically correct analysis model can be optimized to make the implementation more efficient, but an optimized system is more obscure and less likely to be reusable in another context.
- The designer must strike an appropriate balance between efficiency and clarity.

During design optimization, the designer must perform the following tasks,

1) Add Redundant Associations for Efficient Access
   - During analysis, it is undesirable to have redundancy in association network because redundant associations do not add any information.
   - During design, we evaluate the structure of the object model for an implementation.

For that, we have to answer the following questions:

* Is there a specific arrangement of the network that would optimize critical aspects of the completed system?
* Should the network be restructured by adding new associations?
* Can existing associations be omitted?

- The associations that were useful during analysis may not form the most efficient network when the access patterns and relative frequencies of different kinds of access are considered.
- In cases where the number of hits from a query is low because only a fraction of objects satisfy the test, we can build an index to improve access to objects that must be frequently retrieved.

- Analyze the use of paths in the association network as follows:
- Examine each operation and see what associations it must traverse to obtain its information.
- Note which associations are traversed in both directions, and which are traversed in a single direction only, the latter can be implemented efficiently with one way pointers.

For each operation note the following items:

- How often is the operation called? How costly is to perform?
- What is the "fan-out" along a path through the network? Estimate the average count of each "many" association encountered along the path.
  - Multiply the individual fan-outs to obtain the fan-out of the entire path; which represents the number of accesses on the last class in the path.
  - Note that "one" links do not increase the fan-out, although they increase the cost of each operation slightly, don't worry about such small effects.

- What is the fraction of "hits" on the final class , that is , objects that meets selection criteria (if any) and is operated on?
  - If most objects are rejected during the traversal for some reason, then a simple nested loop may be inefficient at finding target objects.
  - Provide indexes for frequent, costly operations with a low hit ratio because such operations are inefficient to implement using nested loops to traverse a path in the network.

2)Rearranging Execution Order for Efficiency

- After adjusting the structure of the object model to optimize frequent traversal, the next thing to optimize is the algorithm itself.
- Algorithms and data structures are directly related to each other, but we find that usually the data structure should be considered first.
- One key to algorithm optimization is to eliminate dead paths as early as possible. Sometimes the execution order of a loop must be inverted.

3)Saving Derived Attributes to Avoid Recomputation:

- Data that is redundant because it can be derived from other data can be "cached" or stored in its computed form to avoid the overhead of recomputing it.
- The class that contains the cached data must be updated if any of the objects that it depends on are changed.
- Derived attributes must be updated when base values change. There are 3 ways to recognize when an update is needed:

- **Explicit update**:
- Each attribute is defined in terms of one or more fundamental base objects.
- The designer determines which derived attributes are affected by each change to a fundamental attribute and inserts code into the update operation on the base object to explicitly update the derived attributes that depend on it.

Periodic Recomputation: Base values are updated in bunches.

- Recompute all derived attributes periodically without recomputing derived attributes after each base value is changed.
- Recomputation of all derived attributes can be more efficient than incremental update because some derived attributes may depend on several base attributes and might be updated more than once by incremental approach.
- Periodic recomputation is simpler than explicit update and less prone to bugs.
- periodic recomputation is not practical because too many derived attributes must be recomputed when only a few are affected.

Active values: An active value is a value that has dependent values.

- Each dependent value registers itself with the active value, which contains a set of dependent values and update operations.
- An operation to update the base value triggers updates all dependent values, but the calling code need not explicitly invoke the updates.
- It provides modularity.

Implementation of Control

- The designer must refine the strategy for implementing the state – event models present in the dynamic model.
- As part of system design, you will have chosen a basic strategy for realizing dynamic model, during object design.
- There are three basic approaches to implement the dynamic model:
  - Using the location within the program to hold state(Procedure driven System)
  - Direct implementation of a state machine mechanism(Event-driven System)
  - Using concurrent tasks

1.State as Location within a Program:

- This is the traditional approach to representing control within a program.
- The location of control within a program implicitly defines the program state.
- Any finite state machine can be implemented as a program.
- Each state transition corresponds to an input statement.
- After input is read, the program branches depending on the input event received.
- Each input statement need to handle any input value that could be received at that point.

One technique of converting state diagram to code is as follows:

- Identify the main control path. Beginning with the initial state, identify a path through the diagram that corresponds to the normally expected sequence of events. Write the name of states along this path as a linear sequence of events.This becomes a sequence of statements in the program.
- Identify alternate paths that branch off the main path and rejoin it later. These become conditional statements in the program.

- Identify backward paths that branch off the main loop and rejoin it earlier .These become loops in program. If multiple backward paths that do not cross, they become nested loops. Backward paths that cross do not nest and can be implemented with goto if all else fails, but these are rare.
- The states and transitions that remain correspond to exception conditions. They can be handled using error subroutines , exception handling supported by the language , or setting and testing of status flags. In the case of exception handling, use goto statements.

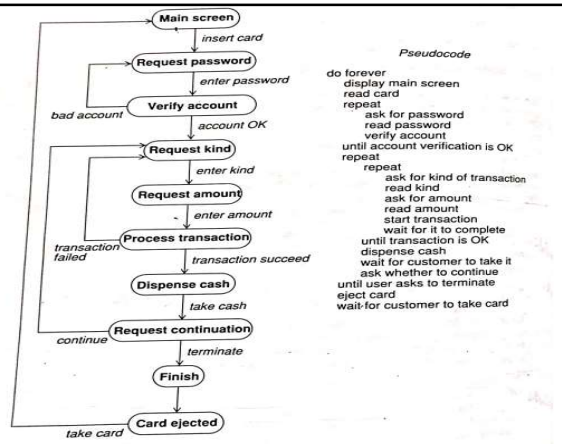

Figure 10.7 ATM control

2.State machine engine

- The most direct approach to control is to have some way of explicitly representing and executing state machine.
- For example, state machine engine class helps execute state machine represented by a table of transitions and actions provided by the application.
- Each object instance would contain its own independent state variables but would call on the state engine to determine next state and action.
- This approach allows you to quickly progress from analysis model to skeleton prototype of the system by defining classes from object model state machine and from dynamic model and creating “stubs” of action routines

- A stub is a minimal definition of function /subroutine without any internal code.
- Thus if each stub prints out its name , technique allows you to execute skeleton application to verify that basic flow of control is correct.
- This technique is not so difficult.

3.Control as Concurrent Tasks

- An object can be implemented as task in programming language /operating system.
- It preserves inherent concurrency of real objects.
- Events are implemented as inter task calls using facilities of language/operating system.
- Concurrent C++/Concurrent Pascal support concurrency.
- Major Object Oriented languages do not support concurrency.

Adjustment of Inheritance

The definitions of classes and operations can often be adjusted to increase the amount of inheritance.

The designer should:

- *Rearrange and adjust classes and operations to increase inheritance.*
- *Abstract Common Behavior Out of groups of classes*
- *Use Delegation to Share behavior when inheritance is semantically valid.*

1.Rearrange classes and operations

- Sometimes the same operation is defined across several classes and can easily be inherited from a common ancestor, but more often operations in different classes are similar but not identical.
- By slightly modifying the definitions of the operations or the classes , the operations can often be made to match so that they can be covered by a single inherited operation.
- Before inheritance can be used , each operation must have the same interface and the types of arguments and results.
- If the signatures match, then the operations must be examined to see if they have the same semantics.

The following kinds of adjustments can be used to increase the chance of inheritance.

- Some operations may have fewer arguments than others .The missing arguments can be added but ignored.
- Some operations may have few arguments because they are special cases of more general arguments .Implement the special operations by calling the general operation with appropriate parameter values.

- Similar attributes in different classes may have different names. Give the attributes the same name and move them to a common ancestor class. Then operations that access the attributes will match better.
- Any operation may be defined on several different classes in a group but undefined on the other classes. Define it on the common ancestor class and define it as no-operation on the classes that do not care about it.

2.Abstracting Out Common Behavior

- Reexamine the object model looking for commonality between classes.
- New classes and operations are often added during design.
- If a set of operations / attributes seems to be repeated in two classes , it is possible that the two classes are specialized variations of the same thing.
- When common behavior has been recognized , a common superclass can be created that implements the shared features , specialized features in subclass.
- This transformation of the object model is called abstracting out a common superclass / common behavior .

- Usually the superclass is abstract, meaning no direct instances.
- Sometimes a superclass is abstracted even when there is only one subclass; here there is no need of sharing.
- Superclass may be reusable in future projects.
- It is an addition to the class library. When a project is completed , the reusable classes should be collected, documented and generalized so that they may be used in future projects.
- Another advantage of abstract superclasses other than sharing and reuse is modularity.
- Abstract superclasses improve the extensibility of a software product.
- It helps in the configuration management of software maintenance and distribution.

3.Use Delegation to Share Implementation

- Sometimes programmers use inheritance as an implementation technique with no intention of guaranteeing the same behavior.
- Sometimes an existing class implements some of the behavior that we want to provide in a newly defined class, although in other respects the two classes are different.
- The designer may inherit from the existing class to achieve part of the implementation of the new class. This can lead to problems –unwanted behavior.
- One object can selectively invoke the desired functions of another using delegation rather than inheritance.
- Delegation consists of catching an operation on one object and sending it to another object that is part of or a related to the first object.

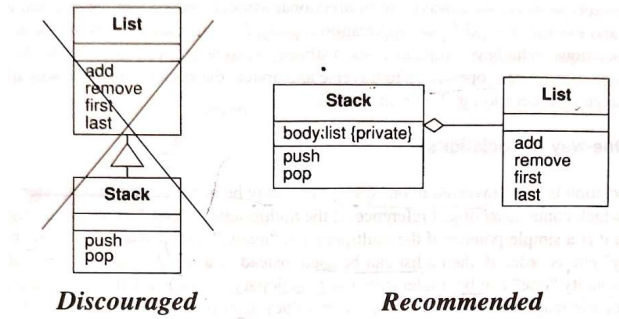

Figure 10.8 Alternative implementations of a Stack using inheritance (left) and delegation (right)

- Every instance of Stack includes a private instance of List.
- Stack: push operation delegates to list by calling its last and add operations to add the element at the end of list, and the pop operation has a same implementation using last and remove operations.
- The ability to corrupt stack by adding or removing arbitrary elements is hidden from client of the Stack class.

Design of Associations

- During object design phase, we must formulate a strategy for implementing all associations in the object model.
- We can either choose a global strategy for implementing all associations uniformly , or a particular technique for each association.

1.Analyzing Association Traversal

- Associations are inherently bidirectional.
- If association in your application is traversed in one direction, their implementation can be simplified.
- The requirements on your application may change, you may need to add a new operation later that needs to traverse the association in reverse direction.
- For prototype work, use bidirectional association so that we can add new behavior and expand /modify.
- In the case of optimization work, optimize some associations.

2.One-way association

- If an association is only traversed in one direction it may be implemented as pointer.
- If multiplicity is 'many' then it is implemented as a set of pointers.
- If the "many" is ordered , use list instead of set .
- A qualified association with multiplicity one is implemented as a dictionary object(A dictionary is a set of value pairs that maps selector values into target values.)
- Qualified association with multiplicity "many" are rare.(it is implemented as dictionary set of objects).

3.Two-way associations

- Many associations are traversed in both directions, although not usually with equal frequency.
- There are three approaches to their implementation:

1. Implement as an attribute in one direction only and perform a search when a backward traversal is required. This approach is useful only if there is great disparity in traversal frequency and minimizing both the storage cost and update cost are important.

2.Implement as attributes in both directions.

- It permits fast access, but if either attribute is updated then the other attribute must also be updated to keep the link consistent .
- This approach is useful if accesses outnumber updates.
- Below Fig.shows the implementation of two-way association using pointers.

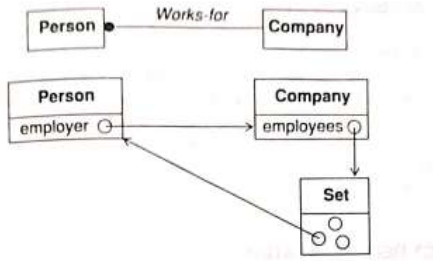

Figure 10.10 Implementation of two-way association using pointers

- Implement as a distinct association object independent of either class.
- An association object is a set of pairs of associated objects stored in a single variable size object.
- An association object can be implemented using two dictionary object one for forward direction and other for reverse direction.

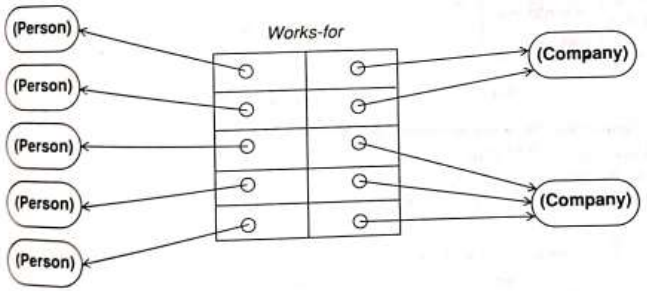

Figure 10.11 Implementation of association as an object

4.Link Attributes

- If an association has link attributes, then its implementation depends on the multiplicity .
- If it is a one-one association , link attribute is stored in any one of the classes involved.
- If it is a many-one association, the link attribute can be stored as attributes of many object ,since each "many" object appears only once in the association.
- If it is a many-many association, the link attribute can't be associated with either object; implement association as distinct class where each instance is one link and its attributes.

Object Representation

- Implementing objects is mostly straight forward, but the designer must choose when to use primitive types in representing objects and when to combine groups of related objects.
- Classes can be defined in terms of other classes, but eventually everything must be implemented in terms of built-in-primitive data types, such as integer strings, and enumerated types.
- Eg:consider the implementation of a social security number within an employee object.

- It can be implemented as an attribute or a separate class.
- Defining a new class is more flexible but often introduces unnecessary indirection.
- The designer must often choose whether to combine groups of related objects.

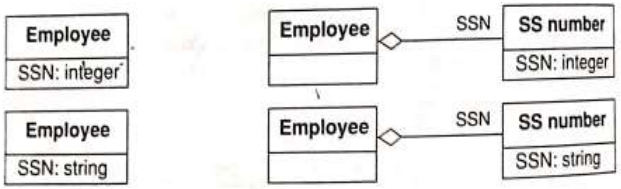

Figure 10.12 Alternative representations for an attribute

Physical Packaging

- Programs are made of discrete physical units that can be edited, compiled, imported, or otherwise manipulated.
- In C and Fortran the units are source files.
- In Ada, it is packages.
- In object oriented languages, there are various degrees of packaging.
- In any large project, careful partitioning of an implementation into packages is important to permit different persons to cooperatively work on a program.

- Packaging involves the following issues:

1.Hiding internal information from outside view.

- One design goal is to treat classes as 'black boxes' , whose external interface is public but whose internal details are hidden from view.
- Hiding internal information permits implementation of a class to be changed without requiring any clients of the class to modify code.
- Additions and changes to the class are surrounded by "fire walls" that limit the effects of any change so that changes can be understood clearly.
- Trade off between information hiding and optimization activities.

- During analysis , we are concerned with information hiding.
- During design , the public interface of each class must be defined carefully.
- The designer must decide which attributes should be accessible from outside the class.
- These decisions should be recorded in the object model by adding the annotation {private} after attributes that are to be hidden , or by separating the list of attributes into 2 parts.
- A method on a class could traverse all the associations of the object model to locate and access another object in the system .
- This is appropriate during analysis

- During design we try to limit the scope of any one method.
- We need to define the bounds of visibility that each method requires.
- Each operation should have a limited knowledge of the entire model, including the structure of classes, associations and operations.
- The fewer things that an operation knows about, the less likely it will be affected by any changes.
- The fewer operations know about details of a class, the easier the class can be changed if needed.

The following design principles help to limit the scope of knowledge of any operation:

- Allocate to each class the responsibility of performing operations and providing information that pertains to it.
- Call an operation to access attributes belonging to an object of another class
- Avoid traversing associations that are not connected to the current class.
- Define interfaces at high level of abstraction as possible.

- Hide external objects at the system boundary by defining abstract interface classes, that is, classes that mediate between the system and the raw external objects.
- Avoid applying a method to the result of another method, unless the result class is already a supplier of methods to the caller.
- Instead consider writing a method to combine the two operations.

2. Coherence of entities.

- One important design principle is coherence of entities.
- An entity ,such as a class , an operation , or a module , is coherent if it is organized on a consistent plan and all its parts fit together toward a common goal.
- It shouldn't be a collection of unrelated parts.
- A method should do one thing well .
- A single method should not contain both policy and implementation.
- A policy is the making of context dependent decisions.
- Implementation is the execution of fully specified algorithms.

- Policy involves making decisions, gathering global information, interacting with outside world and interpreting special cases.
- Policy methods contain input output statements, conditionals and accesses data stores.
- It doesn't contain complicated algorithms but instead calls various implementation methods.
- An implementation method does exactly one operation without making any decisions, assumptions, defaults or deviations .
- All information is supplied as arguments , so the argument list may be long.

- Separating policy and implementation increase reusability.
- Therefore implementation methods don't contain any context dependency.
- They are simple and consists of high level decisions and calls on low-level methods.
- A class shouldn't serve too many purposes.

3.Constructing physical modules.

- During analysis and system design phases we partitioned the object model into modules.
- The initial organization may not be suitable for final packaging of system implementation
- New classes added to existing module or layer or separate module.
- Modules should be defined so that interfaces are minimal and well defined.
- Connectivity of object model can be used as a guide for partitioning modules.

- Classes that are closely connected by associations should be in the same module.
- Loosely connected classes should be grouped in separate modules.
- Classes in a module should represent similar kinds of things in the application or should be components of the same composite object.
- Try to encapsulate strong coupling within a module.
- Coupling is measured by number of different operations that traverse a given association.
- The number expresses the number of different ways the association is used, not the frequency.

Documenting Design Decisions

- The above design decisions must be documented when they are made, or you will become confused.
- This is especially true if you are working with other developers.
- It is impossible to remember design details for any non trivial software system, and documentation is the best way of transmitting the design to others and recording it for reference during maintenance.

- The design document is an extension of the Requirements Analysis Document.
- -> The design document includes revised and much more detailed description of the object model-both graphical and textual.
- ->Additional notation is appropriate for showing implementation decisions, such as arrows showing the traversal direction of associations and pointers from attributes to other objects.

- -> Functional model will also be extended. It specifies all operation interfaces by giving their arguments, results, input-output mappings and side effects.
- -> Dynamic model – if it is implemented using explicit state control or concurrent tasks then the analysis model or its extension is adequate.
- If it is implemented by location within program code, then structured pseudocode for algorithms is needed.
- Keep the design document different from analysis document .
- The design document includes many optimizations and implementation artifacts.

- It helps in validation of software and for reference during maintenance.
- Traceability from an element in analysis to element in design document should be straightforward.
- Therefore the design document is an evolution of analysis model and retains same names.